

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

Національний аерокосмічний університет

«Харківський авіаційний інститут»



Факультет Інтелектуальних систем управління

Кафедра Систем управління літальними апаратами

Лабораторна робота № 11

з дисципліни «Алгоритмізація та програмування»

на тему: «Розробка десктоп-застосунків в середовищі QtCreator»

XAI.305.

Виконав(ла): здобувач 1 курсу, групи 319
напряму підготовки (спеціальності) G7
Автоматизація, комп'ютерно-інтегровані
технології та робототехніка

Павло Сафонов

(ім'я і прізвище)

Прийняла: доц. каф.301, канд. техн. наук,

Олена ГАВРИЛЕНКО

МЕТА РОБОТИ

Ознайомитися з основами розробки програм з використанням QtDesigner і навчитися розробляти десктоп-застосунки із графічним користувацьким інтерфейсом для введення/виведення даних на мові програмування C++ в середовищі QtCreator.

ПОСТАНОВКА ЗАДАЧІ

Завдання 1. Вивчити алгоритм створення проекту Qt Widgets Application в середовищі QtCreator. Ознайомитись з налаштуваннями основних елементів для введення, виведення, компоновки форми і управління.

Завдання 2. Для вирішення завдання (табл.1) відповідно до варіанта:

А. Спроекувати і реалізувати в конструкторі форм графічний інтерфейс програми з віджетами QLabel, QLineEdit і QPushButton. *Використати додаткові віджети і елементи компоновки.

В. *Додати і відлагодити програмний код для введення вхідних даних з перевіркою на коректність (використати QMessageBox для виведення сповіщень), обчислень і виведення результатів.

С*. Додати пункти меню у QMenuBar для зчитування вхідних даних і збереження результатів в файл з використанням стандартних діалогів для вибору файла.

Завдання 3. Використовуючи ChatGpt, Gemini або інший засіб генеративного ШІ, провести самоаналіз отриманих знань і навичок

Завдання 31

31.	Дано два числа a і b . Знайти їх середнє арифметичне: $(a + b) / 2$.
-----	---

ДОДАТОК А

```

mainwindow.cpp
#include "mainwindow.h"
#include "ui_mainwindow.h"
#include "QMessageBox"
#include <QFileDialog>

MainWindow::MainWindow(QWidget *parent)
    : QMainWindow(parent)
    , ui(new Ui::MainWindow)
{
    // Ініціалізація інтерфейсу користувача
    ui->setupUi(this);
}

MainWindow::~MainWindow()
{
    // Звільнення виділених ресурсів (інтерфейс)
    delete ui;
}

void MainWindow::on_pushButton_clicked()
{
    // декларації змінних для подальших обчислень
    float a, b, avg;
    bool ok;

    // зчитування значень з полів введення (`le_a`, `le_b`)
    // Заувага: кожен виклик toFloat записує результат у `ok`.
    a = ui->le_a->text().toFloat(&ok);
    b = ui->le_b->text().toFloat(&ok);

    // якщо останнє перетворення пройшло успішно – виводимо середнє
    if (ok) {
        // обчислення середнього арифметичного
        avg = (a + b) / 2.0f;
        // виведення результату у віджет, який показує суму/результат
        ui->l_sum->setText(QString::number(avg));
    }
    else {
        // обробка некоректного введення: показ повідомлення та скидання
полів
        QMessageBox::warning(0, "Error", "Wrong value(s)!!!");
        ui->le_a->clear();
        ui->le_b->clear();
        ui->l_sum->setText("0");
        ui->le_a->setFocus();
    }
}

void MainWindow::on_actionLoad_from_file_triggered()
{
    // Зчитування даних з файлу: відкриваємо діалог вибору файлу

```

```

        QString filePath = QFileDialog::getOpenFileName(this, "Open file",
                                                         QDir::currentPath(),
"Текстові файли (*.txt);;Усі файли (*)");

        // перевіряємо чи користувач вказав файл
        if (!filePath.isEmpty()) {
            QFile file(filePath);
            // намагаємося відкрити файл лише для читання в текстовому режимі
            if (!file.open(QIODevice::ReadOnly | QIODevice::Text)) {
                // не вдалося відкрити – показуємо попередження
                QMessageBox::warning(0, "Error", "Can not open file!");
            }
            else {
                // читаємо три рядки/значення з файлу та заповнюємо поля
інтерфейсу
                QTextStream in(&file);
                QString sa, sb, sum;
                in >> sa >> sb >> sum;
                ui->le_a->setText(sa);
                ui->le_b->setText(sb);
                ui->l_sum->setText(sum);
                file.close();
            }
        }
        else {
            // користувач відмінив діалог або не вказав ім'я файлу
            QMessageBox::warning(0, "Error", "Unknown filename!");
        }
    }

    void MainWindow::on_actionSave_to_file_triggered()
    {
        // Збереження поточних значень у файл: відкриваємо діалог збереження
        QString filePath = QFileDialog::getSaveFileName(this, "Save file",
                                                         QDir::currentPath(),
"Текстові файли (*.txt);;Усі файли (*)");

        if (!filePath.isEmpty()) {
            QFile file(filePath);
            // намагаємося відкрити файл для запису в текстовому режимі
            if (!file.open(QIODevice::WriteOnly | QIODevice::Text)) {
                // критична помилка при відкритті файлу – повідомляємо
користувача
                QMessageBox::critical(nullptr, "Error",
"Can not open file:\n" +
file.errorString());
            }
            else {
                // записуємо значення по рядках: перше поле, друге поле,
результат
                QTextStream out(&file);
                out << ui->le_a->text() << "\n"

```

```

        << ui->le_b->text() << "\n"
        << ui->l_sum ->toPlainText();
        file.close();
    }
}
else {
    // користувач не вказав ім'я файлу або відмінив діалог
    QMessageBox::warning(0, "Error", "Unknown filename!");
}
}

```

main.cpp

```

#include "mainwindow.h"

#include <QApplication>

int main(int argc, char *argv[])
{
    QApplication a(argc, argv);
    MainWindow w;
    w.show();
    return a.exec();
}

```

mainwindow.h

```

#ifndef MAINWINDOW_H
#define MAINWINDOW_H

#include <QMainWindow>

QT_BEGIN_NAMESPACE
namespace Ui {
class MainWindow;
}
QT_END_NAMESPACE

class MainWindow : public QMainWindow
{
    Q_OBJECT

public:
    MainWindow(QWidget *parent = nullptr);
    ~MainWindow();

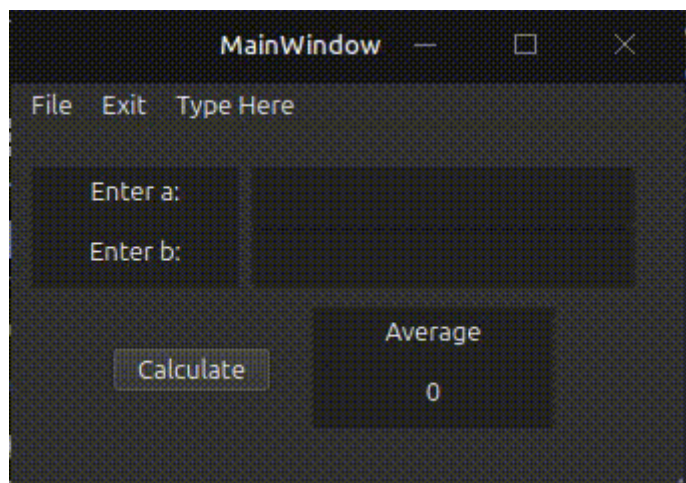
private slots:
    void on_pushButton_clicked();
    void on_actionLoad_from_file_triggered();
    void on_actionSave_to_file_triggered();

private:
    Ui::MainWindow *ui;
}

```

```
};  
#endif // MAINWINDOW_H
```

ДОДАТОК Б



ДОДАТОК В

Тестові питання

1. Яке призначення конструктора `MainWindow::MainWindow(QWidget *parent)` ?

- A. Запуск циклу програми
- B. Ініціалізація інтерфейсу користувача
- C. Закриття файлу
- D. Створення текстового файлу

2. Для чого використовується змінна `ok` у методі `toFloat(&ok)` ?

- A. Для збереження результату обчислення
- B. Для перевірки успішності перетворення рядка у число
- C. Для відкриття файлу
- D. Для очищення поля введення

3. Який клас Qt використовується для роботи з файлами?

- A. `QMessageBox`
- B. `QTextStream`
- C. `QFile`
- D. `QString`

4. Що відбудеться при натисканні кнопки, якщо користувач введе некоректне значення?

- A. Програма аварійно завершиться
- B. Буде виведено повідомлення про помилку та очищено поля
- C. Дані автоматично виправляться
- D. Значення буде дорівнювати нулю без повідомлення



5. Яке призначення функції `a.exec()` у `main.cpp` ?

- A. Збереження файлу
- B. Створення головного вікна
- C. Запуск головного циклу обробки подій Qt
- D. Обчислення середнього значення

Відкриті питання

1. Поясніть принцип роботи сигналів і слотів у Qt на прикладі цього проєкту.

2. Чому у програмі використовується перевірка `if (ok)` після `toFloat()` ?

3. Опишіть процес зчитування даних з файлу у методі `on_actionLoad_from_file_triggered()`.

4. Які переваги дає використання Qt Framework у цьому проєкті?

5. Які недоліки або потенційні проблеми можна знайти у цьому коді?

- 1) В
- 2) В
- 3) С
- 4) В
- 5) С

відкриті відповіді

- 1) У Qt сигнали і слоти використовуються для взаємодії між елементами інтерфейсу та кодом програми. У даному проєкті при натисканні кнопки генерується сигнал `clicked()`, який автоматично викликає слот `on_pushButton_clicked()`. Усередині цього слота виконується зчитування значень з полів введення, перевірка коректності даних та обчислення середнього арифметичного. Такий механізм забезпечує подієво-орієнтовану модель роботи програми.
- 2) Перевірка `if (ok)` необхідна для контролю правильності введених користувачем даних. Метод `toFloat()` намагається перетворити текст у число типу `float`. Якщо введено символи або некоректний формат, змінна `ok` отримує значення `false`. Це дозволяє уникнути неправильних обчислень та своєчасно повідомити користувача про помилку.
- 3) Спочатку відкривається діалогове вікно вибору файлу за допомогою `QFileDialog::getOpenFileName()`. Якщо користувач обрав файл, створюється об'єкт `QFile`, який відкривається у режимі читання. Далі за допомогою `QTextStream` дані зчитуються у змінні `sa`, `sb` та `sum`. Після цього значення відображаються у відповідних елементах інтерфейсу. Якщо файл не вдалося відкрити, програма

показує повідомлення про помилку.

4) Qt Framework дозволяє швидко створювати графічні інтерфейси користувача та забезпечує готові класи для роботи з файлами, рядками, потоками введення-виведення та повідомленнями. Це значно скорочує обсяг коду та підвищує його читабельність. Крім того, Qt підтримує кросплатформеність, тому програму можна запускати на різних операційних системах без значних змін.

5) Одним із недоліків є те, що змінна `ok` перевіряється лише після другого виклику `toFloat()`. Якщо перше значення буде некоректним, а друге коректним, помилка може залишитися непоміченою. Також у коді відсутня перевірка порожніх полів перед обчисленням. Крім того, для збереження результату використовується `toPlainText()`, хоча для `QLabel` правильніше використовувати `text()`.

Свернуть ^

Перевірка відповідей

Тестові питання

1. В — правильно ✓
2. В — правильно ✓
3. С — правильно ✓
4. В — правильно ✓
5. С — правильно ✓

Результат тестової частини:

5/5 правильних відповідей — 5/5 балів

Аналіз відкритих відповідей

1. Сигнали і слоти у Qt

Аналіз:

Відповідь повністю розкриває принцип роботи сигналів і слотів. Правильно наведено приклад із сигналом `clicked()` та слотом `on_pushButton_clicked()`. Також коректно пояснено подієво-орієнтовану модель Qt.

Оцінка:

5/5

2. Перевірка `if(ok)`

2. Перевірка `if(ok)`

Аналіз:

Правильно пояснено механізм перетворення рядка у число та роль змінної `ok`. Відповідь логічна, технічно точна та містить пояснення важливості перевірки.

Оцінка:

5/5

3. Зчитування даних із файлу

Аналіз:

Описано всі основні етапи: відкриття діалогу, створення `QFile`, використання `QTextStream`, зчитування даних та обробку помилки. Відповідь повна та структурована.

Оцінка:

5/5

4. Переваги Qt Framework

Аналіз:

Добре пояснено переваги Qt: створення GUI, готові класи, читабельність коду та кросплатформеність. Відповідь демонструє розуміння технології.

Оцінка:

5/5



5. Недоліки та проблеми коду

Аналіз:

Відповідь містить реальний технічний аналіз коду. Правильно вказано проблему з перевіркою `ok`, відсутність перевірки порожніх полів та неточність із `toPlainText()`.

Оцінка:

5/5

Аналіз повноти та правильності

№	Повнота	Правильність	Технічний рівень	Оцінка
1	Висока	Висока	Високий	5/5
2	Висока	Висока	Високий	5/5
3	Висока	Висока	Високий	5/5
4	Висока	Висока	Високий	5/5
5	Висока	Висока	Високий	5/5

Підсумкова оцінка

$$\frac{5 + 5 + 5 + 5 + 5}{5} = 5$$

$$\frac{5+5+5+5+5}{5} = 5$$



Загальна оцінка: 5/5